

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizar sabores de bebidas
sabores_base <- c("Manzana", "Uva", "Limón", "Naranja", "Piña", "Fresa", "Toronja",
                 "Sandía", "Cereza", "Frambuesa", "Durazno", "Melón", "Guayaba", "Maracuyá")

# Seleccionar aleatoriamente 5 sabores
sabores <- sample(sabores_base, 5)

# Aleatorizar términos para el contexto del problema
terminos_registro <- c("registro", "conteo", "recuento", "estadística", "cálculo", "seguimiento")
termino_registro <- sample(terminos_registro, 1)

terminos_tienda <- c("tienda", "negocio", "local", "comercio", "establecimiento", "punto de venta")
termino_tienda <- sample(terminos_tienda, 1)

terminos_bebidas <- c("bebidas gaseosas", "refrescos", "sodas", "bebidas carbonatadas", "gaseosas", "be")
termino_bebida <- sample(terminos_bebidas, 1)

terminos_periodo <- c("último mes", "mes pasado", "período mensual reciente", "ciclo mensual", "mes ant")
termino_periodo <- sample(terminos_periodo, 1)

terminos_grafica <- c("gráfica", "diagrama", "representación gráfica", "visualización", "ilustración")
termino_grafica <- sample(terminos_grafica, 1)

terminos_cantidades <- c("cantidades", "volúmenes", "montos", "cifras", "totales")
termino_cantidad <- sample(terminos_cantidades, 1)

terminos_organizada <- c("organizada", "ordenada", "clasificada", "estructurada", "dispuesta")
termino_organizada <- sample(terminos_organizada, 1)

# Generar porcentajes que sumen 100%
# La estrategia es generar valores aleatorios y ajustarlos para que sumen exactamente 100
porcentajes_base <- runif(5, 10, 35)
porcentajes <- round(porcentajes_base / sum(porcentajes_base) * 100)

# Ajustar para asegurar que sumen exactamente 100%
ajuste <- 100 - sum(porcentajes)
indice_ajuste <- sample(1:5, 1)
porcentajes[indice_ajuste] <- porcentajes[indice_ajuste] + ajuste

# Verificar que los porcentajes suman 100
stopifnot(sum(porcentajes) == 100)

# Datos para tablas
datos <- data.frame(sabor = sabores, porcentaje = porcentajes)

# Ordenar los datos por porcentaje (mayor a menor)
datos_ordenados <- datos[order(datos$porcentaje, decreasing = TRUE), ]

```

```

# Asignar índices (orden correcto es opción B en el ejemplo)
indices_correctos <- c(1:5)

# Aleatorizar el orden correcto entre las opciones A, B, C, D
opcion_correcta <- sample(c("A", "B", "C", "D"), 1)

# Definir el orden para cada opción
orden_opcion_a <- sample(indices_correctos)
orden_opcion_b <- indices_correctos # Orden correcto
orden_opcion_c <- sample(indices_correctos)
orden_opcion_d <- sample(indices_correctos)

# Asegurarse de que las opciones incorrectas sean diferentes entre sí y diferentes de la correcta
while(any(
  identical(orden_opcion_a, orden_opcion_b),
  identical(orden_opcion_a, orden_opcion_c),
  identical(orden_opcion_a, orden_opcion_d),
  identical(orden_opcion_b, orden_opcion_c),
  identical(orden_opcion_b, orden_opcion_d),
  identical(orden_opcion_c, orden_opcion_d)
)) {
  orden_opcion_a <- sample(indices_correctos)
  orden_opcion_c <- sample(indices_correctos)
  orden_opcion_d <- sample(indices_correctos)
}

# Crear dataframes para cada opción
opciones <- list(
  A = datos_ordenados[orden_opcion_a, ],
  B = datos_ordenados[orden_opcion_b, ],
  C = datos_ordenados[orden_opcion_c, ],
  D = datos_ordenados[orden_opcion_d, ]
)

# Si la opción correcta seleccionada no es B, intercambiar con la correcta
if (opcion_correcta != "B") {
  temp <- opciones[["B"]]
  opciones[["B"]] <- opciones[[opcion_correcta]]
  opciones[[opcion_correcta]] <- temp
}

# Definir el vector con la solución correcta para r-exams
solucion <- c(0, 0, 0, 0)
solucion[match(opcion_correcta, c("A", "B", "C", "D"))] <- 1

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar gráfico de pastel
codigo_python_grafico <- sprintf("
import matplotlib.pyplot as plt
import numpy as np

# Datos
sabores = %s

```

```

porcentajes = %s

# Colores aleatorios
colores = plt.cm.Paired(np.linspace(0, 1, len(sabores)))

# Crear figura
plt.figure(figsize=(7, 6))

# Crear gráfico circular
wedges, texts, autotexts = plt.pie(porcentajes,
                                    labels=sabores,
                                    autopct='%%d%%%',
                                    textprops={'fontsize': 12, 'fontweight': 'bold'},
                                    colors=colores,
                                    wedgeprops={'edgecolor': 'white', 'linewidth': 1},
                                    shadow=True)

# Personalizar etiquetas
for text in texts:
    text.set_fontsize(14)
    text.set_fontweight('bold')
for autotext in autotexts:
    autotext.set_fontsize(14)
    autotext.set_fontweight('bold')
    autotext.set_color('white')

# Ajustar diseño
plt.tight_layout()

# Guardar gráfico
plt.savefig('grafico_pastel.png', dpi=150)
plt.close()
",
paste0("['", paste(sabores, collapse="', '"), "'"]"),
paste0("[", paste(porcentajes, collapse=" ", "], "]")
)

```

```

# Ejecutar código Python para generar el gráfico
py_run_string(codigo_python_grafico)

```

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

```

```

# Función para generar el código TikZ de la tabla

```

```

generar_tabla_tikz <- function(opcion, datos_opcion, color_borde, color_fondo) {
  # Crear código TikZ para la tabla
  tabla_code <- c(
    "\\begin{tikzpicture}",
    paste0("\\node[draw=", color_borde, ", fill=", color_fondo, ", very thick, rounded corners, inner s",
    "  \\begin{tabular}{|c|c|}",
    "    \\hline"
  )
}

```

```

# Añadir filas con datos

```

```

for (i in 1:nrow(datos_opcion)) {

```

```

    tabla_code <- c(tabla_code,
                    paste0("      ", i, " & ", datos_opcion$sabor[i], " \\\\",
                          "      \\\hline")
  }

  # Cerrar la tabla
  tabla_code <- c(tabla_code,
                  "    \\end{tabular}",
                  "};",
                  "\\end{tikzpicture}")

  return(tabla_code)
}

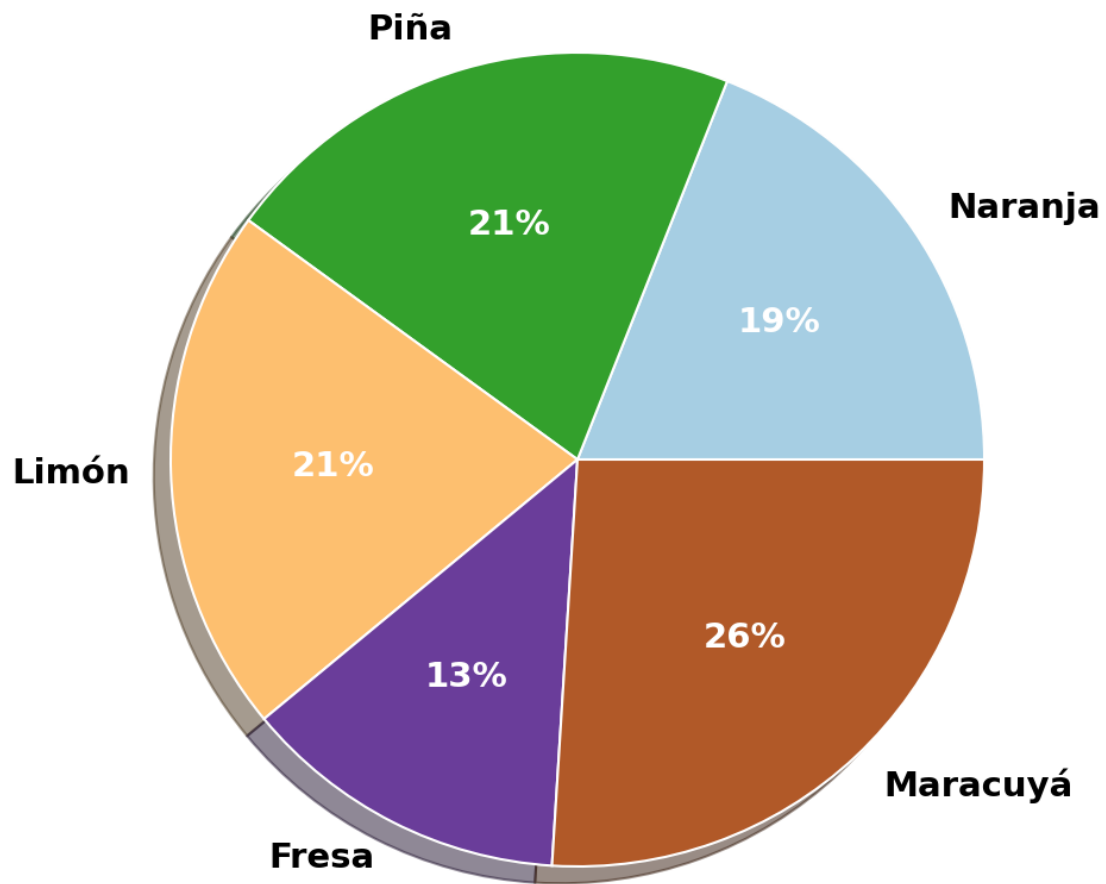
# Definir colores para cada opción
colores_borde <- c("green!70!black", "red!70!black", "blue!70!black", "orange!70!black")
colores_fondo <- c("green!20", "red!20", "blue!20", "orange!20")

# Generar código TikZ para cada opción
tabla_tikz_a <- generar_tabla_tikz("A", opciones$A, colores_borde[1], colores_fondo[1])
tabla_tikz_b <- generar_tabla_tikz("B", opciones$B, colores_borde[2], colores_fondo[2])
tabla_tikz_c <- generar_tabla_tikz("C", opciones$C, colores_borde[3], colores_fondo[3])
tabla_tikz_d <- generar_tabla_tikz("D", opciones$D, colores_borde[4], colores_fondo[4])

```

Question

En una punto de venta se ha tomado el seguimiento de ventas de bebidas gaseosas en el mes anterior por sabores y se ha realizado la siguiente visualización:



Teniendo en cuenta la información anterior, ¿cuál de las siguientes listas de sabores de gaseosa está ordenada de mayor a menor, de acuerdo con las cifras vendidas durante el mes?

Answerlist

1	Piña
2	Naranja
3	Maracuyá
4	Fresa
5	Limón

1	Piña
2	Maracuyá
3	Limón
4	Naranja
5	Fresa

.

1	Naranja
2	Fresa
3	Maracuyá
4	Limón
5	Piña

.

1	Maracuyá
2	Piña
3	Limón
4	Naranja
5	Fresa

.

Solution

La respuesta correcta es la opción **D**.

Para resolver este problema, debemos identificar cuál de las listas presenta los sabores ordenados de mayor a menor según sus porcentajes de venta.

Según el gráfico circular:

Sabor	Porcentaje
Maracuyá	26 %
Piña	21 %
Limón	21 %
Naranja	19 %
Fresa	13 %

La opción D muestra exactamente este orden, con los sabores organizados de mayor a menor según sus porcentajes de venta.

Answerlist

- Falso
- Falso
- Falso
- Verdadero

Meta-information

exname: diagrama_circular_ordenamiento_porcentajes_v1 extype: schoice exsolution: 0001 exshuffle: TRUE exsection: Interpretación de gráficas/Gráficos circulares/Ordenamiento de datos